

This is a **high-quality, industry-relevant project**, but it is also **ambitious**.

Is it doable? **Yes**, for a team of 3, but **only if you scope it correctly right now**. If you try to implement a full-blown standard YOLO model from scratch in Verilog without prior experience, you will struggle.

Since your immediate hurdle is **Assignment 1 (Due Feb 16th)**, here is a breakdown of why this is doable and how you can "hack" the deadline to get full marks on the first phase.

1. The Feasibility Check

- **The Team (3 Members):** This is the perfect size. This project naturally splits into three distinct roles, so no one is stepping on toes:
 - **Member 1 (The Hardware Architect):** Focuses on the PL (Programmable Logic). Writes the HLS/Verilog for the convolution and pooling blocks.
 - **Member 2 (The Software/Integration Engineer):** Focuses on the PS (Processing System). Sets up the Zynq board, runs Linux/Bare-metal, handles OpenCV for image capture, and sends data to the FPGA.
 - **Member 3 (The Model & Verification Lead):** Trains the "Custom 3-layer CNN" in Python/TensorFlow first, extracts the weights, quantizes them (converts float to integer), and writes the testbench to verify the hardware matches the Python results.

2. Strategy for Assignment 1 (Deadline: Feb 16)

You do **not** need working code for Feb 16. You need a solid **plan**. Assignment 1 asks for the "Algorithm description," "Datapath/Control path," and "Fixed-point analysis."

Here is exactly what you should propose to make this deadline manageable:

A. Choose the "Custom 3-layer CNN" (The "Smart" Choice)

Do not commit to "Tiny-YOLO" or "MobileNet" in your documentation yet. Those are complex graphs.

- **The Pitch:** Propose a **LeNet-5 style architecture** or a simple **3-layer CNN** (Conv2D $\rightarrow$ ReLU $\rightarrow$ MaxPool $\rightarrow$ FC).
- **Why:** Drawing the datapath for a 3-layer CNN is easy. Drawing the datapath for YOLO (with its skip connections and complex routing) is a nightmare for a 5-mark assignment due in a few days.
- **Note:** You can always "upgrade" to a better model later, but for Assignment 1, simple is better.

B. The "Hardware Mapping" (Crucial for Marks)

Your document must explicitly state *how* you are splitting the work.

- **PS (Arm Cortex):**

- Captures image from USB/Onboard Camera.
- Resizes image to 32×32 or 64×64 (grayscale).
- Sends image data to FPGA via **AXI-DMA** (This is the keyword your prof wants to see).
- **PL (FPGA Fabric):**
 - Receives pixels.
 - Buffers lines (Line Buffer).
 - Performs Sliding Window Convolution.
 - Sends classification result back to Arm.

C. Fixed-Point Analysis

- This is a requirement in the PDF.
- **The Answer:** State that you will use **INT8 (8-bit integer)** or **Fixed Point (e.g., Q8.8)** quantization.
- **Reasoning:** Floating point is too expensive for simple FPGA logic. INT8 reduces memory usage and speeds up calculation, which fits the "efficiency" requirement.

3. Immediate Action Plan (For the Feb 16 Deadline)

To submit Assignment 1 on time, you need to generate these three artifacts immediately:

1. **Block Diagram:** A high-level view showing the Camera $\rightarrow$ Arm Processor $\rightarrow$ AXI Interconnect $\rightarrow$ CNN Accelerator IP $\rightarrow$ Output.
2. **Datapath Diagram:** Specifically for the **Convolution Unit**.
 - Show: Line Buffers $\rightarrow 3 \times 3$ Window Register $\rightarrow 9$ Multipliers $\rightarrow$ Adder Tree $\rightarrow$ ReLU.
3. **FSM Diagram (Control Path):**
 - States: IDLE $\rightarrow$ LOAD_WEIGHTS $\rightarrow$ RECEIVE_IMAGE $\rightarrow$ COMPUTE_CONV $\rightarrow$ POOLING $\rightarrow$ WRITE_OUTPUT $\rightarrow$ DONE.

Would you like me to help you draft the text for the "Algorithm Description" or describe exactly how the Datapath diagram for the Convolution unit should look?